



PROGRAMAÇÃO



Escola Naval

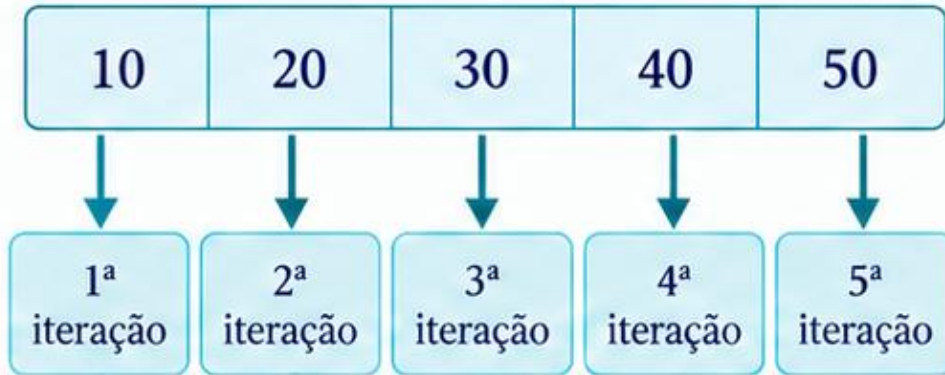


António Gonçalves

AGENDA

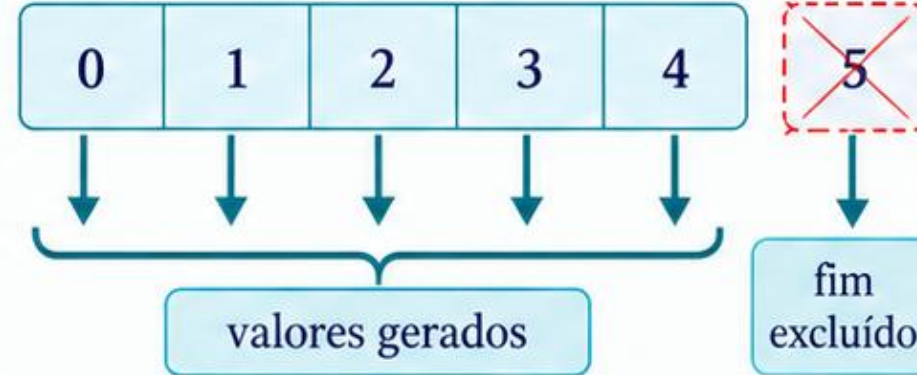
1 for e sequências

Utilizar ciclos for para percorrer elementos de uma sequência, como listas, tuplos e cadeias de caracteres.



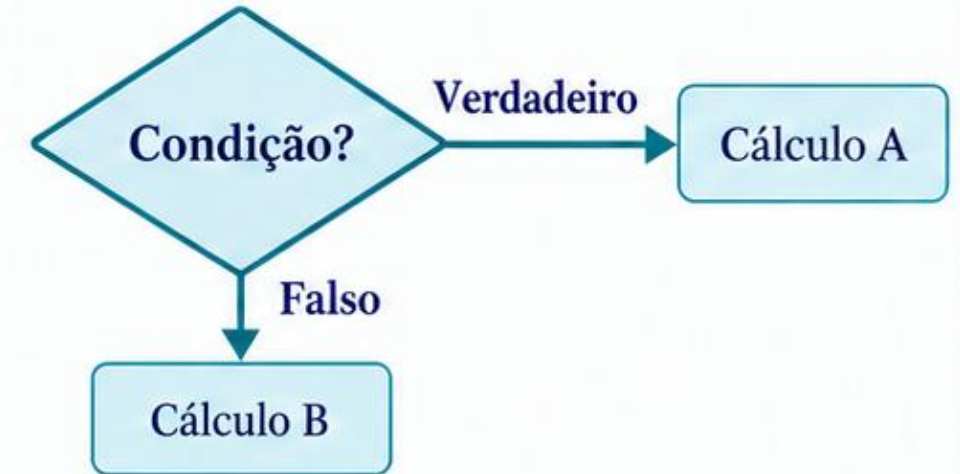
2 range e limites

Usar a função range() para gerar sequências de números, definindo os limites de início, fim e passo.



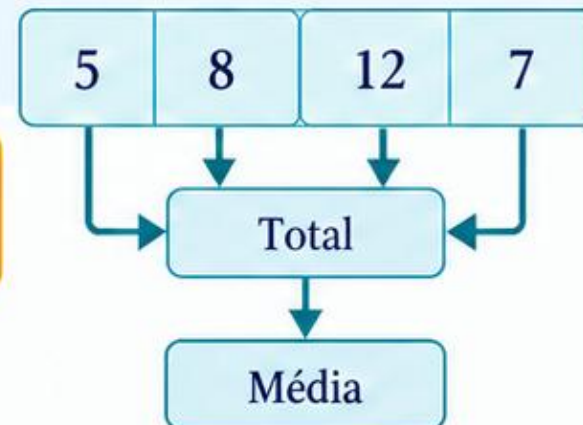
3 Decisões e cálculos

Aplicar decisões condicionais para alterar o fluxo de execução e realizar diferentes cálculos consoante a condição.



4 Totais e médias

Acumular valores num total e calcular a média de um conjunto de números.



5 Ciclos aninhados e controlo de execução

Utilizar ciclos dentro de ciclos e instruções de controlo para gerir a execução, como interromper ou saltar iterações.



O ciclo for

Definição: Percorre, um a um, os elementos de uma sequência ou de outro iterável.

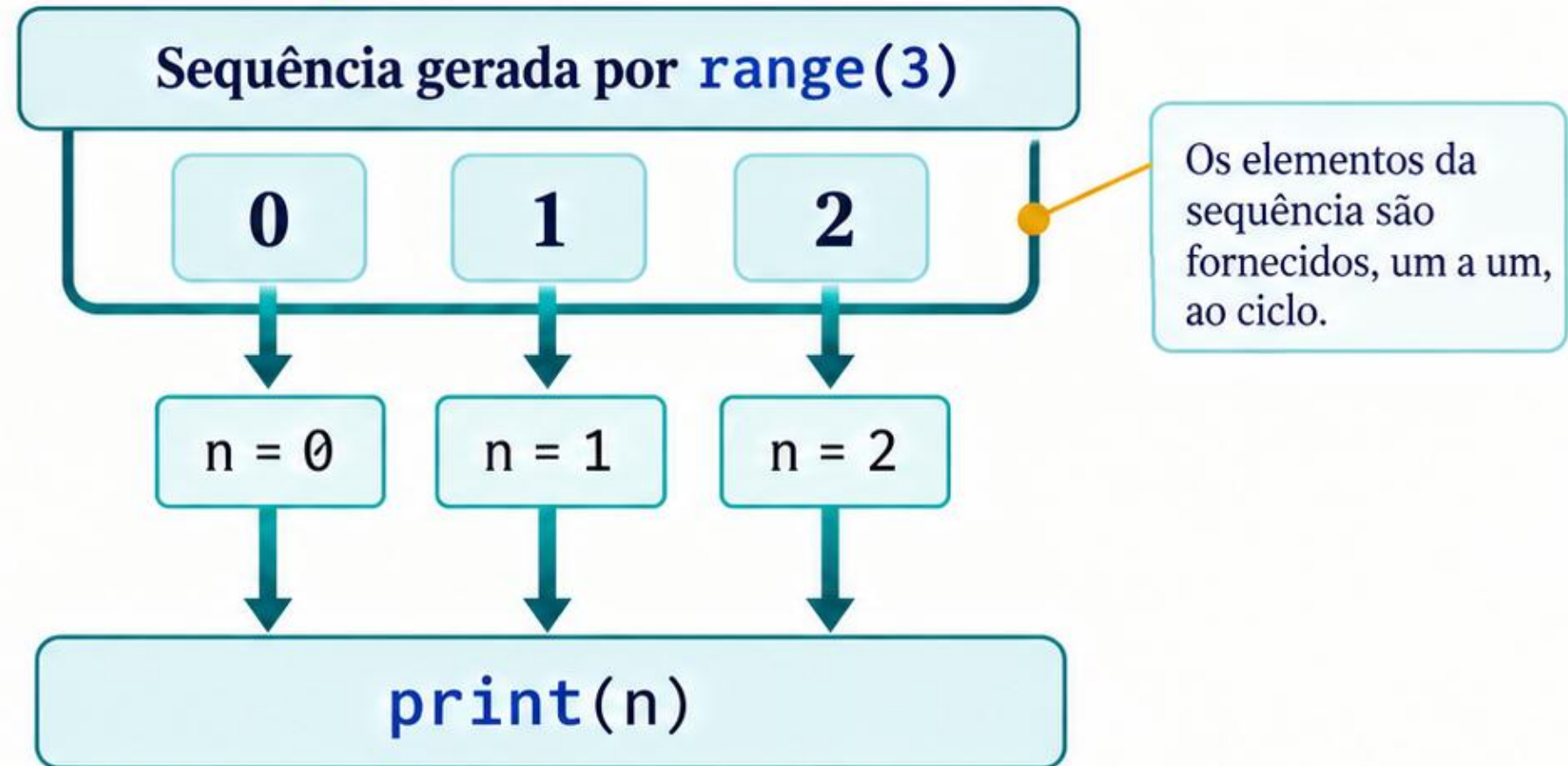
Exemplo:

```
for n in range(3):  
    print(n)
```

Resultado:

```
0  
1  
2
```

Com **range()**, podemos controlar uma contagem.



Os elementos da sequência são fornecidos, um a um, ao ciclo.



Com **range()**, podemos controlar uma contagem.

ANATOMIA DO for

for: palavra-chave que inicia o ciclo.

Variável n:
assume, em cada iteração,
um valor da sequência.

in: indica que a variável
vai percorrer uma
sequência de valores.

range(3): gera os valores
0, 1 e 2 (três iterações).

Dois pontos (:):
marcam o início
do bloco do ciclo.

```
for n in range(3):
```

```
{ print(n)  
  print("Fim")
```

Bloco indentado:
instruções executadas
em cada iteração.

Saída no ecrã:

```
0  
1  
2  
Fim
```

**Valores de n
em cada iteração:**

n = 0

n = 1

n = 2

i

Não é necessário escrever $n = n + 1$.

range(fim)

range(5) dá 0, 1, 2, 3, 4.

Começa em 0, avança de 1 em 1 e exclui o limite final.

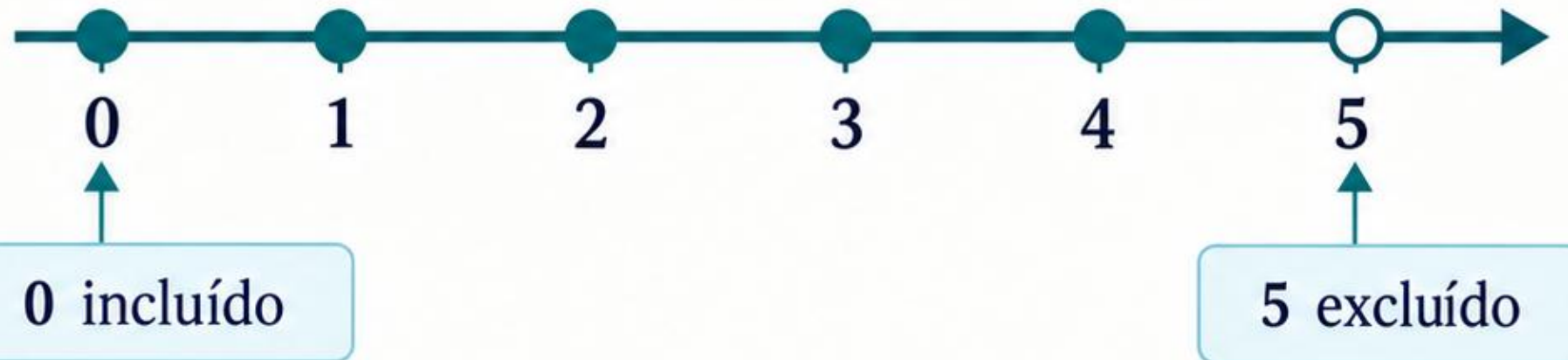
Código em Python

```
for n in range(5):  
    print(n)
```

Saída

0
1
2
3
4

Linha numérica



Valores gerados

0 1 2 3 4

Quantidade

5 valores.

range(início, fim)

O **início** está incluído. O **fim** fica de fora.

range(2, 6)



Gera: 2, 3, 4, 5

range(1, 6)



Gera: 1, 2, 3, 4, 5

O **início** está incluído. O **fim** fica de fora.

Código

```
for n in range(1, 6):  
    print(n)
```

Resultado

1
2
3
4
5

range(início, fim, passo)

A função **range** gera uma sequência de números inteiros, começando em **início**, aumentando de acordo com o **passo**, e terminando antes de **fim** (o valor **fim** é excluído).

início
(incluído)

2

fim
(excluído)

11

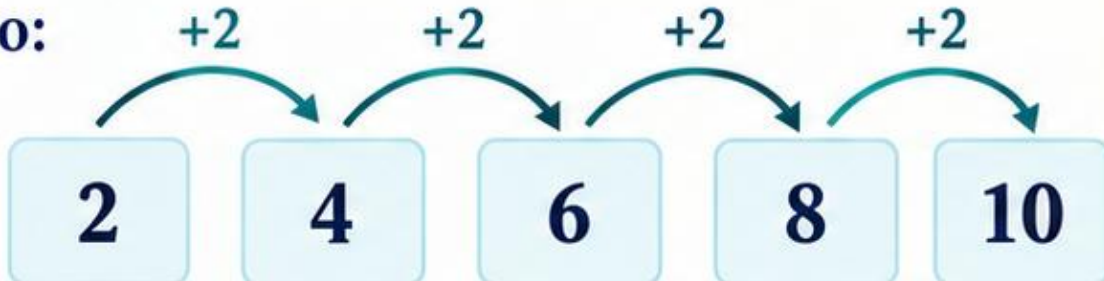
passo
(incremento)

2

Exemplo 1

```
for n in range(2, 11, 2):  
    print(n)
```

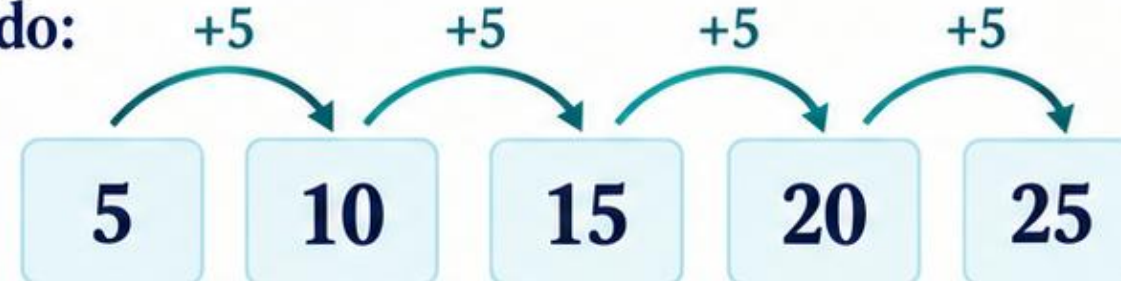
Resultado:



Exemplo 2

```
for n in range(5, 26, 5):  
    print(n)
```

Resultado:



O **passo** define a diferença entre valores consecutivos.

CONTAGENS DECRESCENTES

1

```
list(range(5, 0, -1))
```



2

```
list(range(5, -1, -1))
```



3

```
list(range(10, 0, -2))
```



Código Python	Resultado
<code>list(range(5, 0, -1))</code>	<code>[5, 4, 3, 2, 1]</code>
<code>list(range(5, -1, -1))</code>	<code>[5, 4, 3, 2, 1, 0]</code>
<code>list(range(10, 0, -2))</code>	<code>[10, 8, 6, 4, 2]</code>

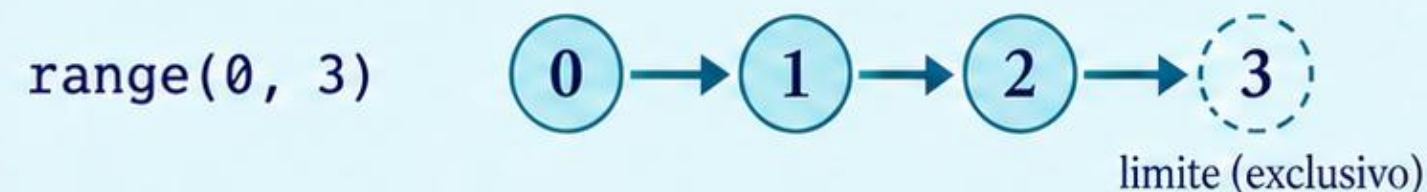
Para diminuir, use passo negativo. O limite final continua excluído.

LIMITES E SEQUÊNCIAS VAZIAS

A função `range()` gera uma sequência de números. Os limites e o passo determinam os valores produzidos.

Expressão	Valores produzidos	Observações
<code>range(3)</code>	0, 1, 2	Início em 0, até 3 (exclusivo).
<code>range(3, 3)</code>	(nenhum valor)	Limites iguais produzem sequência vazia.
<code>range(5, 1)</code>	(nenhum valor)	Passo positivo não alcança o limite.
<code>range(5, 1, -1)</code>	5, 4, 3, 2	Passo negativo regressa do 5 até 1 (exclusivo).

O passo tem de avançar na direção do limite.



Passo zero não é permitido:
`range(1, 5, 0)` provoca **ValueError**.

Num ciclo, o corpo não é executado quando a sequência é vazia.

```
for x in range(3, 3):  
    print(x)
```

→ Não é produzida qualquer saída.

PREVER OS VALORES DE range

Exercício: para cada instrução com range, indique:

- 1 o valor inicial (início);
- 2 o valor final (excluído);
- 3 o valor do passo (incremento);
- 4 o número de iterações (quantos valores são gerados);
- 5 os valores produzidos pelo range.

```
for x in range(4):  
    print(x)  
  
for x in range(2, 6):  
    print(x)  
  
for x in range(1, 10, 2):  
    print(x)  
  
for x in range(10, 2, -2):  
    print(x)
```

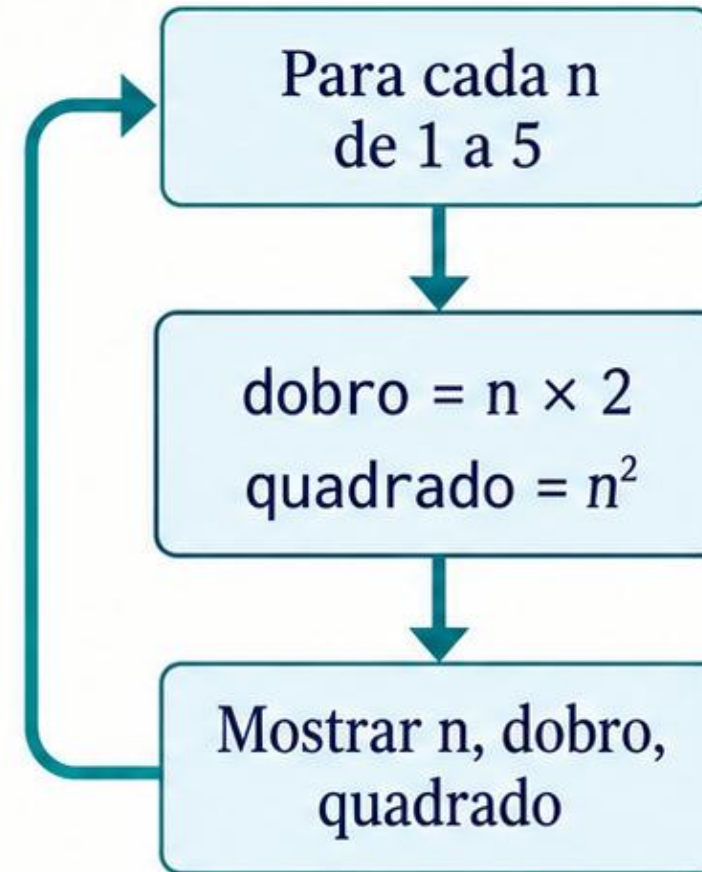
Instrução	Início	Final (excluído)	Passo	N.º de iterações	Valores gerados
range(4)	0	4	1	4	0, 1, 2, 3
range(2, 6)	2	6	1	4	2, 3, 4, 5
range(1, 10, 2)	1	10	2	5	1, 3, 5, 7, 9
range(10, 2, -2)	10	2	-2	4	10, 8, 6, 4

O valor final é excluído, o passo determina o incremento e o número de iterações corresponde ao número de valores gerados.

CÁLCULOS EM CADA ITERAÇÃO

Código em Python

```
for n in range(1,6):  
    dobro = n * 2  
    quadrado = n ** 2  
    print(n, dobro, quadrado)
```



<i>n</i>	dobro	quadrado
1	2	1
2	4	4
3	6	9
4	8	16
5	10	25

1 Iteração

A variável *n* assume, sucessivamente, os valores de 1 a 5.

2 Cálculos

Em cada iteração são calculados o dobro ($n \times 2$) e o quadrado (n^2) do valor de *n*.

3 Resultados

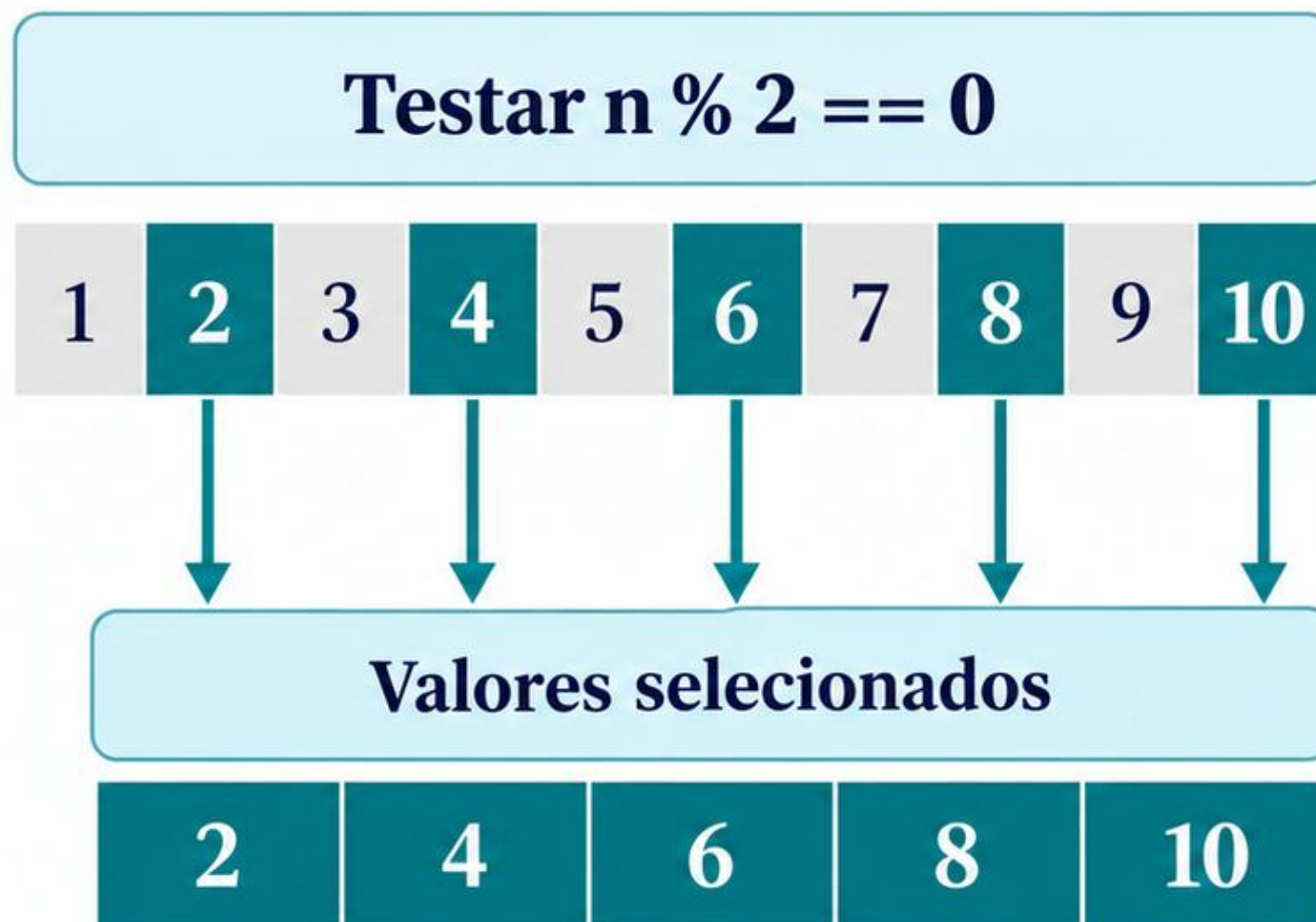
Em cada iteração é apresentado o valor de *n*, o seu dobro e o seu quadrado.

for com decisões

O **for** percorre os valores.
O **if** seleciona os que cumprem a condição.

Código Python

```
for n in range(1, 11):  
    if n%2==0:  
        print(n)
```



Iterações do ciclo (n)									
1	2	3	4	5	6	7	8	9	10

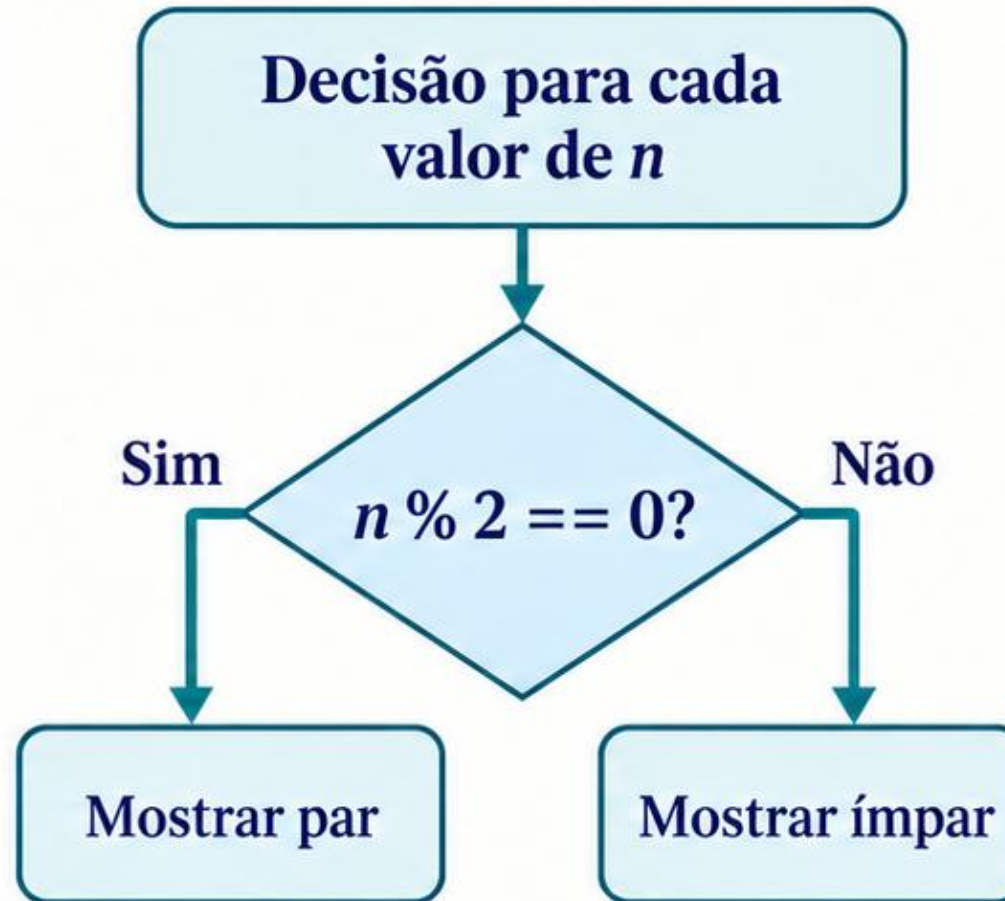
Valores impressos				
2	4	6	8	10

Total de iterações: **10**
Total de impressões: **5**

CLASSIFICAR EM CADA ITERAÇÃO

Código em Python

```
for n in range(1,6):  
    if n%2==0:  
        print(n, "é par")  
    else:  
        print(n, "é ímpar")
```



Saída	Resultado
1	1 é ímpar
2	2 é par
3	3 é ímpar
4	4 é par
5	5 é ímpar

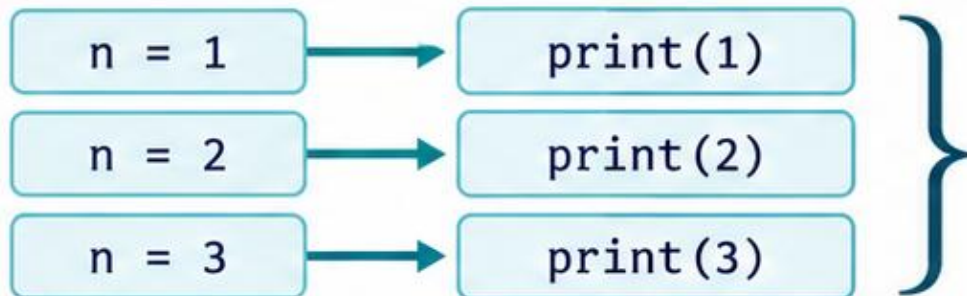
A decisão é repetida para cada valor.

for ou while?

O ciclo **for** itera sobre uma sequência de valores (por exemplo, um intervalo), executando o bloco de instruções uma vez por cada elemento.

```
for n in range(1, 4):  
    print(n)
```

Execução:

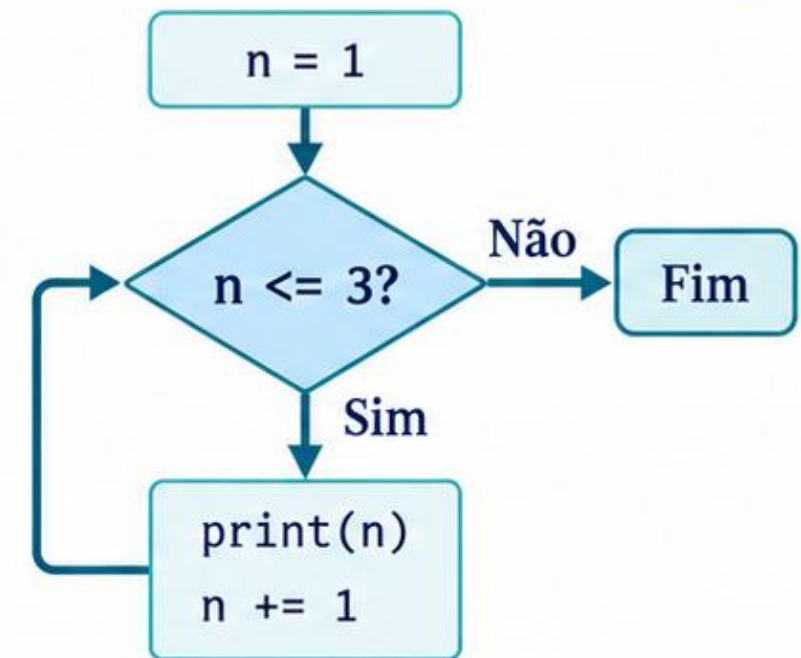


Saída:

1
2
3

O ciclo **while** repete a execução de um bloco de instruções enquanto a condição for verdadeira. A condição é avaliada antes de cada iteração.

```
n = 1  
while n <= 3:  
    print(n)  
    n += 1
```



Quando usar for?

- Para iterar sobre sequências (listas, tuplos, intervalos, etc.).
- Quando o número de iterações é conhecido.



Quando usar while?

- Para repetir enquanto uma condição for verdadeira.
- Útil, por exemplo, em validação de dados ou quando a condição de paragem depende da execução.



Não existe uma regra absoluta.

Se souber o número de iterações (por exemplo, um intervalo), o **for** é o mais simples e adequado. Caso contrário, o **while** permite maior controlo através de uma condição.

CONTADOR E ACUMULADOR

O **contador** regista quantos valores são processados e o **acumulador** soma os valores.
Inicialize antes do ciclo.

```
quantidade = 0
total = 0

for valor in (10, 5, 8):
    quantidade += 1
    total += valor
```

Valores

10

5

8

Processar cada valor

```
quantidade += 1
total += valor
```

Resultado

```
quantidade = 3
total = 23
```

Traço da execução

Passo	Valor	quantidade	total
Inicial	–	0	0
1	10	1	10
2	5	2	15
3	8	3	23

Actualização do acumulador

```
total += valor
```

≡

```
total = total + valor
```

A instrução `total += valor` tem o mesmo efeito que `total = total + valor`.

Inicialize antes do ciclo.

Defina `quantidade = 0` e `total = 0` antes de começar a processar os valores.

Somar valores introduzidos

Código em Python

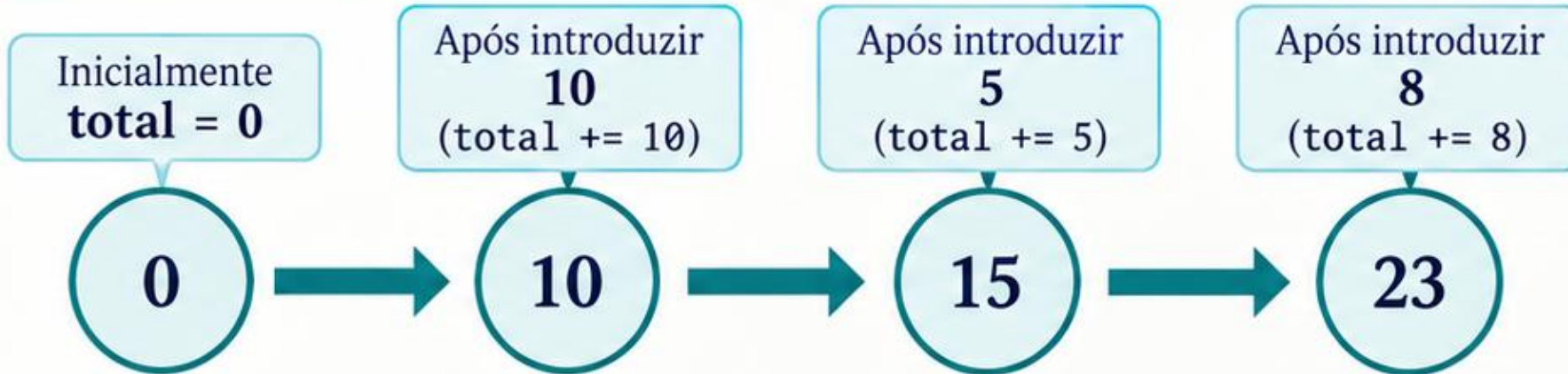
```
total = 0
for n in range(3):
    valor = float(input("Valor: "))
    total += valor
print("Total:", total)
```

Execução (exemplo)

Valor: 10
Valor: 5
Valor: 8
Total: 23.0

O comando **print** está fora do ciclo, por isso apresenta apenas o total final.

Evolução do valor de total



Iteração	Valor introduzido	total (após soma)
1	10	10
2	5	15
3	8	23

Se o comando **print** estivesse dentro do ciclo, seriam apresentados os totais parciais (10, 15 e 23).

O ciclo repete três vezes (**range(3)**) e soma os três valores introduzidos pelo utilizador.

ACUMULAÇÃO E CONTAGEM CONDICIONAIS

Ler todos os valores, processar apenas os positivos.

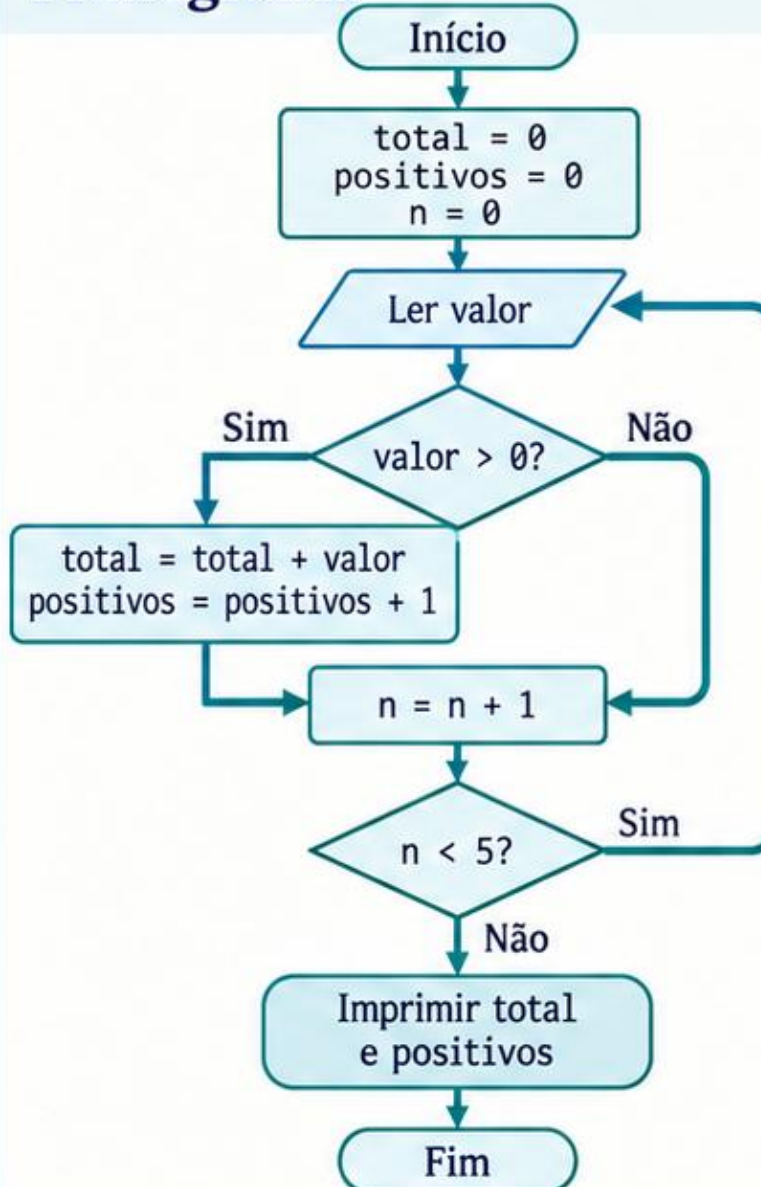
Código em Python

```
total = 0
positivos = 0
for n in range(5):
    valor = int(input("Valor: "))
    if valor > 0:
        total += valor
        positivos += 1

print("Total:", total)
print("Positivos:", positivos)
```

A variável **total** acumula a soma dos valores positivos e a variável **positivos** conta quantos foram lidos.

Fluxograma



Exemplo de execução

Iteração	Entrada (valor)	Processa?	total (após)	positivos (após)
1	10	Sim	10	1
2	-2	Não	10	1
3	0	Não	10	1
4	5	Sim	15	2
5	8	Sim	23	3

Saída obtida

Total: 23
Positivos: 3

O valor zero não é processado, pois não é positivo.

MÉDIA DE NOTAS

Somar durante o ciclo. Dividir depois do ciclo.
Consideram-se válidas as notas introduzidas.

Código em Python

```
total = 0
for n in range(4):
    nota = float(input("Nota: "))
    total += nota
media = total / 4
print("Média:", media)
```

Entradas do utilizador

Nota: 12
Nota: 14
Nota: 16
Nota: 18



Saída produzida

Média: 15.0

Execução (valores após cada iteração)

Iteração (n)	Nota introduzida	Valor de total (após somar)
0	12	12
1	14	26
2	16	42
3	18	60
Após o ciclo:		media = total / 4 media = 60 / 4 = 15.0

O programa soma as quatro notas durante o ciclo e calcula a média depois do ciclo.

CICLOS ANINHADOS

Código Python

```
for linha in range(1,3):  
    for coluna in range(1,4):  
        print(linha,coluna)
```

Ciclo aninhado: um ciclo (interno) é executado completamente para cada iteração do ciclo (externo).

Total de execuções: $2 \times 3 = 6$.
O ciclo interno repete 3 vezes para cada valor de linha.

Coordenadas produzidas

linha = 1 →	(1, 1)	(1, 2)	(1, 3)
linha = 2 →	(2, 1)	(2, 2)	(2, 3)
	↑	↑	↑
	coluna = 1	coluna = 2	coluna = 3

Rasto de execução

Iteração (linha)	Iteração (coluna)	Saída impressa
1	1	1 1
1	2	1 2
1	3	1 3
2	1	2 1
2	2	2 2
2	3	2 3

DECISÃO EM CICLOS ANINHADOS

Percorrem-se todas as combinações de linha e coluna de uma grelha 3 × 3. A instrução de decisão verifica se linha é igual a coluna e, nesse caso, imprime o par correspondente.

```
for linha in range(1,4):
    for coluna in range(1,4):
        if linha==coluna:
            print(linha,coluna)
```

Saída produzida

1 1
2 2
3 3

Grelha 3 × 3

		coluna		
		1	2	3
linha	1	(1,1)	(1,2)	(1,3)
	2	(2,1)	(2,2)	(2,3)
	3	(3,1)	(3,2)	(3,3)

Todas as combinações

Iteração	linha	coluna	linha==coluna?	Imprime?
1	1	1	Sim	Sim
2	1	2	Não	Não
3	1	3	Não	Não
4	2	1	Não	Não
5	2	2	Sim	Sim
6	2	3	Não	Não
7	3	1	Não	Não
8	3	2	Não	Não
9	3	3	Sim	Sim

Número total de combinações:

9

Número de impressões (condição verdadeira):

3

A condição `linha==coluna` seleciona apenas os elementos da diagonal principal.

Os ciclos aninhados permitem percorrer todas as posições e a decisão seleciona a diagonal.

break: terminar o ciclo

Código em Python

```
for n in range(1,10):  
    if n==5:  
        break  
    print(n)
```

Resultado da execução

1
2
3
4

break termina o ciclo mais interno em que se encontra.

Iteração	Valor de n	Condição n == 5	Ação executada	Saída
1	1	Falsa	executa print(n)	1
2	2	Falsa	executa print(n)	2
3	3	Falsa	executa print(n)	3
4	4	Falsa	executa print(n)	4
5	5	Verdadeira	executa break (termina o ciclo)	–
6	6	–	não processado	–
7	7	–	não processado	–
8	8	–	não processado	–
9	9	–	não processado	–

Enquanto **n** for diferente de 5, a instrução **print(n)** é executada.

Quando **n** é igual a 5, a instrução **break** é executada e o ciclo termina imediatamente, antes de executar **print(n)**.

Os valores 6, 7, 8 e 9 não são processados, pois o ciclo já terminou.

CONTINUE: AVANÇAR À PRÓXIMA ITERAÇÃO

A instrução **continue** ignora o restante do corpo da iteração atual e avança para a próxima iteração do ciclo.

Exemplo

```
for n in range(1,7):
    if n == 3:
        continue
    print(n)
```

Saída

1
2
4
5
6

Rastreio da execução

Iteração (n)	n == 3?	Ação	Valor impresso
1	Não	Executa print(n)	1
2	Não	Executa print(n)	2
3	Sim	Executa continue	—
4	Não	Executa print(n)	4
5	Não	Executa print(n)	5
6	Não	Executa print(n)	6

A instrução **continue** ignora o restante do corpo da iteração atual.

Comparação com break

break

- 1 Termina imediatamente o ciclo.
- 2 O controlo sai do ciclo.
- 3 As restantes iterações não são executadas.

continue

- 1 Ignora o restante corpo da iteração atual.
- 2 Avança para a próxima iteração.
- 3 O ciclo continua a ser executado até ao fim.

EXERCÍCIO INTEGRADO E SOLUÇÃO

Enunciado:

Ler cinco valores inteiros, somar os valores positivos, contar quantos são positivos e calcular a média dos positivos, se existir.

Solução em Python:

```
total = 0
quantidade = 0
for n in range(5):
    valor = int(input("Valor: "))
    if valor > 0:
        total += valor
        quantidade += 1

if quantidade > 0:
    print("Média:", total / quantidade)
else:
    print("Sem positivos")
```

Teste 1: [10, -2, 0, 5, 15]

Iteração	Valor lido	É positivo?	total	quantidade
1	10	Sim	10	1
2	-2	Não	10	1
3	0	Não	10	1
4	5	Sim	15	2
5	15	Sim	30	3

Resultado:

Média: 10.0

Soma dos positivos:

30

Quantidade:

3

Teste 2: [0, -1, -2, 0, -3]

Iteração	Valor lido	É positivo?	total	quantidade
1	0	Não	0	0
2	-1	Não	0	0
3	-2	Não	0	0
4	0	Não	0	0
5	-3	Não	0	0

Resultado:

Sem positivos

Soma dos positivos:

0

Quantidade:

0

O algoritmo soma apenas os valores positivos e calcula a média, se existir.

SÍNTESE E ESCOLHA DO CICLO

for (iterável)

Percorre, sequencialmente, todos os elementos de um objeto iterável.

```
for item in sequencia:  
    # bloco
```

range()

Gera uma sequência de inteiros, incluindo o início e excluindo o fim.

```
for i in range(inicio,  
               fim, passo):  
    # bloco
```

while

Repete enquanto a condição for verdadeira.

```
while condicao:  
    # bloco
```

Contador

Variável que conta o número de iterações.

```
contador = 0  
for _ in sequencia:  
    contador += 1
```

Acumulador

Variável que acumula um resultado (por exemplo, uma soma).

```
total = 0  
for valor in sequencia:  
    total += valor
```

Nidificação

Um ciclo dentro de outro; o ciclo interior repete para cada iteração do ciclo exterior.

```
for x in exterior:  
    for y in interior:  
        # bloco
```

break

Termina imediatamente o ciclo mais próximo.

```
if condicao:  
    break
```

continue

Passa para a próxima iteração do ciclo mais próximo.

```
if condicao:  
    continue
```

1 Que valores são percorridos?

O ciclo **for** percorre cada elemento do iterável, na ordem definida.

Elemento	Valor
1	a
2	b
3	c
4	d
...	...

2 Quais são os limites e o passo?

A função **range(inicio, fim, passo)** gera inteiros, incluindo o início e excluindo o fim.

início	fim	passo	Valores gerados
0	5	1	0, 1, 2, 3, 4
1	10	2	1, 3, 5, 7, 9
3	3	1	(vazia)
5	2	-1	5, 4, 3

3 Que resultados se acumulam?

Um acumulador vai somando os valores processados em cada iteração.

Iteração	Valor	Soma acumulada
1	3	3
2	5	8
3	2	10
4	4	14

4 Como termina a execução?

O ciclo termina quando a condição deixa de ser verdadeira ou quando ocorre um **break**. O **continue** não termina o ciclo; apenas salta para a próxima iteração.

Situação	Efeito
Condição falsa (while)	termina
Fim do iterável (for)	termina
break	termina
continue	próxima iteração